

# The Impact of Large Language Models on Software Development Productivity: A Systematic Review and Critical Analysis

## Abstract

This paper presents a systematic review of the impact of Large Language Models (LLMs) on software development productivity. Since their widespread adoption in late 2022, LLM-powered coding assistants have been posited as revolutionary tools capable of delivering order-of-magnitude productivity gains. However, empirical evidence presents a complex and often contradictory picture. This review synthesizes findings from 37 peer-reviewed studies, industry case studies, and controlled experiments to analyze the multifaceted effects of LLMs across the software development lifecycle. We ground our analysis in established productivity frameworks, such as SPACE and DORA metrics, to provide a holistic assessment beyond simple output measures. Our findings reveal that while LLMs demonstrably accelerate specific tasks like boilerplate code generation and information retrieval, they also introduce significant risks, including degraded code quality, security vulnerabilities, increased cognitive load for verification, and a potential reduction in team collaboration. Notably, we explore the "productivity paradox," where developers perceive significant efficiency gains even when objective measures indicate a slowdown. We conclude that the impact of LLMs is not a universal enhancement but a complex trade-off, heavily contingent on developer experience, task complexity, codebase size, and the quality of human-AI interaction. The paper identifies critical research gaps, particularly the need for longitudinal and team-based studies, and offers recommendations for practitioners on leveraging LLMs effectively while mitigating their inherent risks.

## Keywords

Large Language Models (LLMs), Software Development Productivity, Generative AI, AI Coding Assistants, GitHub Copilot, Software Engineering, Empirical Study, Systematic Literature Review, SPACE Framework, DORA Metrics.

# **1. Introduction**

## **1.1. The Advent of LLMs in Software Engineering**

The release of generative artificial intelligence (AI) systems such as ChatGPT in late 2022 marked an inflection point for the software industry.<sup>1</sup> Large Language Models (LLMs), particularly those trained on vast repositories of source code, have been rapidly integrated into the software development lifecycle (SDLC) through a new class of tools known as AI coding assistants.<sup>3</sup> The initial hype surrounding these tools has since matured, with the industry conversation shifting from the speculative notion of LLMs replacing human developers to a more pragmatic view of them as partners in an "augmented intelligence" paradigm.<sup>5</sup> This paradigm suggests a future where software engineers and AI-enabled tools share roles, fundamentally altering development workflows and processes.<sup>5</sup>

## **1.2. Background on Large Language Models (LLMs)**

### **1.2.1. Technical Definition**

Technically, LLMs are a category of foundation models built upon deep neural networks, most commonly the transformer architecture.<sup>6</sup> These models are composed of multiple neural network layers enhanced by an "attention mechanism," which enables them to weigh the importance of different parts of the input data, thereby capturing complex relationships

within sequential data like text or code.<sup>7</sup> Trained on immense corpora—often trillions of words from sources like public code repositories and web pages—LLMs learn the statistical patterns of language.<sup>7</sup> Their fundamental operation involves predicting the next "token" (a word or sequence of characters) based on the preceding context, allowing them to generate coherent, contextually relevant, and often human-indistinguishable text and code.<sup>6</sup>

### **1.2.2. Application in Software Development**

In the context of software development, LLMs are primarily deployed as AI-powered coding assistants, often referred to as "AI Pair Programmers".<sup>3</sup> Prominent examples include GitHub Copilot, Cursor, and Tabnine.<sup>10</sup> These tools integrate directly into Integrated Development Environments (IDEs) and assist developers across a wide spectrum of SDLC tasks, including code generation from natural language prompts, code completion, refactoring complex segments, debugging, generating documentation, and creating automated test cases.<sup>13</sup>

## **1.3. The Problem of Measuring Productivity**

### **1.3.1. Defining Software Development Productivity**

The concept of software development productivity has long been a subject of debate and is notoriously difficult to measure.<sup>16</sup> Early, simplistic metrics such as Lines of Code (LOC) produced per unit of time have been widely discredited for failing to account for code quality, complexity, or the value delivered.<sup>16</sup> A more contemporary and robust definition frames productivity as the ratio of the functional value of the software produced to the labor and expense required to produce it.<sup>16</sup> This value-based perspective acknowledges that productivity is not merely about output volume but about the creation of high-quality, maintainable, and effective software that meets user needs.<sup>21</sup>

### **1.3.2. Multi-Dimensional Frameworks**

The inherent complexity of productivity has led to the development of holistic, multi-dimensional frameworks that provide a more nuanced view. This paper leverages two prominent frameworks to structure its analysis:

- **The SPACE Framework:** Developed by researchers from Microsoft and the University of Victoria, the SPACE framework posits that developer productivity is not a single metric but a composite of five key dimensions: **S**atisfaction and well-being, **P**erformance, **A**ctivity, **C**ommunication and collaboration, and **E**fficiency and flow. This framework argues that a balanced view across these dimensions is necessary to avoid optimizing for one aspect at the expense of others.<sup>17</sup>
- **DORA Metrics:** Originating from the DevOps Research and Assessment (DORA) team, these metrics focus on software delivery performance and organizational success. The four key DORA metrics are Deployment Frequency, Lead Time for Changes, Change Failure Rate, and Mean Time to Restore (MTTR). Together, they provide a balanced view of both development velocity and operational stability, capturing the trade-offs between speed and quality.<sup>21</sup>

## 1.4. Importance of the Study

Despite the widespread adoption of and investment in LLM-based tools, a significant gap exists between the prevailing industry hype and the empirical evidence of their impact. Early reports and studies present a conflicting narrative. On one hand, some controlled experiments and enterprise case studies report dramatic increases in speed, with developers completing tasks up to 55% faster.<sup>34</sup> On the other hand, a recent, rigorous randomized controlled trial found that experienced developers were, on average, 19%

*slower* when using state-of-the-art AI assistants.<sup>37</sup> This stark contradiction highlights the urgent need for a systematic synthesis of the available evidence to provide a nuanced, data-driven understanding of how, when, and for whom LLMs truly impact software development productivity.

## 1.5. Objectives and Hypotheses

This paper aims to address this gap through the following objectives and hypotheses:

- **Objective 1:** To systematically review and synthesize the existing empirical literature on the impact of LLM-based coding assistants on software development productivity.
- **Objective 2:** To analyze this impact through the multi-dimensional lens of the SPACE and DORA frameworks, moving beyond a singular focus on speed or output.
- **Objective 3:** To identify the key benefits, risks, and contextual factors that mediate the effectiveness of LLMs in software development.
- **Hypothesis 1 (H1):** The use of LLM assistants is positively correlated with metrics of *Activity* (e.g., commit frequency, pull requests) and developer *Satisfaction*, but has an inconsistent or negative correlation with *Performance* (e.g., code quality, security).
- **Hypothesis 2 (H2):** The productivity impact of LLMs is moderated by developer experience and task complexity, with novice developers and those working on routine tasks experiencing greater relative benefits.

## 2. Review of Literature

### 2.1. The Evolving Landscape of LLM-Powered Development Tools

The market for AI-powered development tools has expanded rapidly since late 2022, moving from a novelty to an integral part of the enterprise software stack.<sup>2</sup> The landscape is dominated by a few key players, each with distinct approaches:

- **GitHub Copilot:** As the most widely adopted tool, Copilot is integrated directly into major IDEs and leverages a suite of models from OpenAI and, more recently, Google's Gemini.<sup>11</sup> Its ubiquity has made it the de facto benchmark in many productivity studies.<sup>34</sup>
- **Cursor:** This tool represents a more "AI-first" approach, built as a standalone IDE forked from the open-source core of VS Code. Its key differentiators include deep indexing of the entire codebase for enhanced context awareness, support for multiple underlying LLMs (from providers like OpenAI and Anthropic), and agentic capabilities that allow it to perform multi-step tasks autonomously.<sup>11</sup>
- **Tabnine:** Tabnine carves a niche by focusing on enterprise needs for privacy and personalization. It allows organizations to train models on their private codebases, ensuring that suggestions align with internal coding standards and patterns while maintaining data confidentiality.<sup>41</sup>

- **Specialized Tools:** Beyond general-purpose assistants, a new wave of specialized tools is emerging. These include platforms like Vercel's v0 for generating user interface components from text prompts and Replit AI for developing and deploying applications entirely within a cloud-native environment.<sup>41</sup>

## 2.2. Empirical Studies on LLM Productivity: A Contradictory Narrative

The empirical evidence on the productivity impact of these tools is deeply divided, creating a contradictory narrative that this review seeks to reconcile.

### 2.2.1. Evidence for Productivity Gains

Several studies, particularly industry-led case studies and early controlled experiments, report substantial productivity improvements. A widely cited study found that developers using GitHub Copilot completed a programming task 55.8% faster than a control group.<sup>36</sup> Similarly, a study using GPT-3 demonstrated a 27% speed improvement for programmers.<sup>36</sup> Enterprise case studies echo these findings. A collaboration between GitHub and Accenture reported that Copilot users saw a 15% increase in pull request merge rates and an 84% increase in successful builds.<sup>35</sup> A separate case study by Faros AI found a 55% reduction in lead time to production for teams using Copilot.<sup>44</sup> A trial within the UK public sector found that users self-reported saving an average of 56 minutes per working day.<sup>43</sup> These quantitative gains are often accompanied by high levels of developer satisfaction and strong perceptions of increased productivity.<sup>35</sup>

### 2.2.2. Evidence for Productivity Losses and the Perception-Reality Gap

In stark contrast, a rigorous randomized controlled trial (RCT) conducted by the research organization METR in July 2025 produced opposite results. The study, involving 16 experienced open-source developers working on real-world tasks from their projects, found that using state-of-the-art AI tools (primarily Cursor with Claude 3.5/3.7 Sonnet) made them, on average, **19% slower**.<sup>37</sup>

Crucially, this study uncovered a significant gap between perception and reality. While objectively slower, the developers using AI tools *believed* they had been approximately 20% faster.<sup>37</sup> This phenomenon, which can be termed the "productivity placebo," suggests that the subjective experience of using AI assistants—characterized by reduced typing and rapid suggestion generation—creates a powerful illusion of efficiency that may not be reflected in end-to-end task completion times. This disconnect is corroborated by other research indicating that while developers consistently report significant productivity gains, objective measurements often show more modest or even negative results.<sup>46</sup>

## 2.3. Mediating Factors Influencing LLM Impact

The conflicting results suggest that the impact of LLMs is not uniform but is heavily mediated by several contextual factors.

- **Developer Experience:** A consistent finding across multiple studies is that the benefits of AI assistants are not evenly distributed across experience levels. Junior developers and novices tend to see the largest relative gains, as LLMs can serve as powerful learning aids, helping to bridge knowledge gaps, explain unfamiliar code, and accelerate onboarding.<sup>40</sup> Conversely, experienced developers, who possess deep existing mental models of a codebase, may be slowed down by the cognitive overhead of verifying, debugging, and correcting AI-generated suggestions that conflict with their expertise.<sup>37</sup>
- **Task Complexity and Type:** LLMs demonstrate the highest efficacy on well-defined, repetitive, or "trivial" tasks, such as generating boilerplate code, writing unit tests, or implementing standard algorithms.<sup>13</sup> Their performance diminishes significantly when faced with complex, novel problems that require deep contextual understanding, abstract reasoning, or innovative architectural solutions.<sup>51</sup>
- **Codebase Size and Language Popularity:** The effectiveness of LLMs is constrained by their "context window"—the amount of information they can consider at one time. Consequently, productivity gains tend to decrease as the size and complexity of the codebase grow, as the model loses the broader project context.<sup>52</sup> Furthermore, performance is typically better for popular programming languages like Python and Java, which are heavily represented in the training data, compared to less common or domain-specific languages.<sup>52</sup>

## 2.4. Challenges and Risks to Code Quality and Security

Beyond speed, a critical aspect of productivity is the quality of the output. Here, the literature raises significant concerns.

- **Code Quality and Maintainability:** While LLMs can help enforce consistent coding styles<sup>4</sup>, they are also prone to generating code that is syntactically correct but logically flawed or inefficient.<sup>53</sup> A 2025 study by Sonar found that over 90% of the issues in LLM-generated code were "code smells"—indicators of poor structure that increase future technical debt and maintenance costs.<sup>54</sup>
- **Security Vulnerabilities:** Security is arguably the most significant risk. An early influential study found that GitHub Copilot generated code containing security vulnerabilities in approximately 40% of scenarios.<sup>53</sup> More recent research confirms this trend, with one study showing that AI-generated code introduced 322% more privilege escalation paths compared to human-written code.<sup>45</sup> These vulnerabilities often stem from the models being trained on outdated code, leading them to suggest deprecated functions with known security flaws, or from their inability to reason about the security implications of the code they produce.<sup>53</sup>
- **New Attack Surfaces:** The integration of LLMs into the SDLC introduces novel security risks, as categorized by the OWASP Top 10 for LLM Applications. These include threats like Prompt Injection, where malicious inputs trick the model into performing unintended actions, and Training Data Poisoning, where the model's knowledge base is corrupted with malicious data.<sup>56</sup>

### 3. Methodology

This paper employs a **Systematic Literature Review (SLR)** methodology to rigorously synthesize and analyze the existing body of work on the impact of LLMs on software development productivity. This approach was chosen for its structured and transparent process, which minimizes bias and ensures a comprehensive overview of the research landscape.

The review was guided by the research objectives and hypotheses outlined in Section 1.5. A comprehensive search was conducted across major academic databases, including IEEE Xplore, ACM Digital Library, and the arXiv pre-print server, as well as grey literature such as industry technical reports and major conference proceedings. The search covered publications from January 2021 to September 2025, using a combination of keywords such as "Large Language Model," "Generative AI," "Software Development," "Developer Productivity,"



"GitHub Copilot," "Empirical Study," and "Case Study."

Studies were included if they presented original empirical data—either quantitative or qualitative—on the use of LLM-based coding assistants by software developers and discussed impacts related to productivity. Opinion pieces, purely theoretical papers, and studies focusing on non-development tasks were excluded. This process yielded 37 primary studies that met the inclusion criteria.<sup>49</sup>

Data from these studies were extracted and synthesized using a thematic analysis approach. To ensure a holistic and structured analysis, the emergent themes were mapped onto the five dimensions of the SPACE framework for developer productivity. This allowed for a systematic examination of the multifaceted impacts of LLMs, explicitly highlighting areas of consensus, contradiction, and research gaps.

## 4. Findings / Results

The synthesized findings from the 37 primary studies are presented below, structured according to the five dimensions of the SPACE framework.

### 4.1. Satisfaction and Well-being

This dimension is where LLM assistants show their most consistent and positive impact. The majority of studies, particularly those incorporating qualitative surveys, report a significant boost in developer satisfaction. In the Accenture enterprise study, 90% of developers reported feeling more fulfilled in their jobs, and 95% said they enjoyed coding more when using GitHub Copilot.<sup>35</sup> This is largely attributed to the reduction of tedious and repetitive tasks, which allows developers to focus on more creative and engaging aspects of their work. Reports indicate that developers experience less mental effort on routine tasks and can more easily maintain a state of "flow".<sup>35</sup>

However, this positive picture is not without its caveats. An emerging concern is the potential for increased cognitive load due to task switching. One study found that teams with high AI adoption handled 47% more pull requests per day, leading to a state of constant "juggling" of parallel workstreams rather than focused, deep work.<sup>45</sup> Furthermore, developers express anxiety about the long-term risks of over-reliance, including the potential for skill erosion or

"deskilling" in fundamental programming concepts.<sup>4</sup>

## 4.2. Performance (Quality, Reliability, Security)

The performance dimension reveals the most critical trade-offs associated with LLM adoption. While some high-level metrics, such as the 84% increase in successful builds reported in the Accenture study, suggest quality improvements, direct analysis of the generated code tells a different and more cautionary story.<sup>35</sup>

The evidence regarding security is particularly alarming. LLMs consistently produce code with significant security flaws. The Sonar study found that for several leading models, over 60% of the vulnerabilities introduced were of "blocker" or high severity, including critical flaws like hard-coded credentials and injection vulnerabilities.<sup>54</sup> This confirms earlier findings that AI assistants can generate vulnerable code in a large percentage of scenarios.<sup>53</sup> A striking finding from the Sonar study illustrates a dangerous dynamic: when a model was updated, its functional correctness pass rate improved by 6.3%, but the percentage of high-severity bugs it produced simultaneously rose by a staggering 93%.<sup>54</sup> This suggests that optimizing for correctness on benchmark tasks may inadvertently increase security risks.

In terms of maintainability, the same study found that over 90% of issues in AI-generated code were "code smells," indicating poor architectural choices that contribute to long-term technical debt.<sup>54</sup> This implies that short-term speed gains may be achieved at the cost of future maintenance burdens.

## 4.3. Activity

Across nearly all studies that measure it, LLM usage is strongly correlated with an increase in raw activity metrics. Reports consistently show a higher volume of pull requests, more frequent code commits, and a greater number of tasks completed per week.<sup>35</sup> While this is often presented as a direct indicator of increased productivity, a deeper analysis reveals a potential "activity trap."

The ease with which LLMs can generate initial code drafts lowers the barrier for developers to begin work on multiple tasks simultaneously, leading to the observed increase in parallel workstreams and pull requests.<sup>13</sup> However, this generated code often requires substantial

human effort for review, debugging, and refactoring to address the quality and security issues discussed previously.<sup>53</sup> This creates a scenario where a developer is engaged in a high volume of

*activity*—creating PRs, responding to feedback, making commits—but this activity is primarily focused on "chaperoning" the AI and correcting its output, rather than efficiently solving the core engineering problem. This dynamic helps to explain the paradoxical outcome of the METR study, where developers were objectively slower at completing a finished, correct task despite likely engaging in more intermediate activities.<sup>37</sup> It suggests that an increase in activity metrics can be a misleading vanity metric, masking underlying inefficiencies and mistaking motion for progress.

#### **4.4. Communication and Collaboration**

The introduction of LLMs into the development workflow appears to have a chilling effect on human-to-human collaboration. A recurring theme in qualitative studies is that developers, especially novices, will often turn to an LLM as their first point of contact for questions or problems, rather than consulting with senior teammates or peers.<sup>49</sup> While this can accelerate individual problem-solving for simple queries, it carries significant risks for team health. This behavior can inhibit the organic transfer of knowledge, reduce opportunities for mentorship, and lead to the formation of knowledge silos, ultimately undermining the collective intelligence of the team.

The impact on the code review process is also mixed. On one hand, LLMs can assist in generating documentation and comments, which can theoretically aid reviewers.<sup>13</sup> On the other hand, the poor quality of AI-generated code often necessitates more intensive reviews. One study found that pull requests containing AI code required 60% more reviewer comments specifically related to security issues, potentially increasing the burden on reviewers and slowing down the overall cycle time.<sup>45</sup>

#### **4.5. Efficiency and Flow**

The analysis of efficiency reveals a critical distinction between micro-level and macro-level impacts. At the micro-efficiency level, LLMs provide clear and undeniable benefits. They dramatically reduce the time developers spend on tasks like searching for syntax examples

online, understanding API usage, and writing repetitive boilerplate code.<sup>49</sup> This reduction in low-level friction is a primary driver of the high perceived productivity and satisfaction reported by developers.

However, at the macro-efficiency level—the end-to-end time required to deliver a complete, correct, and production-ready feature—the picture is far less clear. The workflow with an LLM introduces new, time-consuming steps that are often overlooked in simpler analyses. These include the cognitive overhead of carefully crafting and refining prompts (prompt engineering), the meticulous validation of AI outputs for logical correctness and security flaws, and the debugging of subtle, non-obvious errors introduced by the model.<sup>3</sup> The METR study provides the strongest empirical evidence to date that, for experienced developers working on realistic tasks, the time costs of these new verification and correction steps can outweigh the time savings from automated code generation, resulting in a net loss of end-to-end efficiency.<sup>37</sup>

**Table 1: Summary of Empirical Study Findings on LLM Productivity Impact**

| Study / Report        | Methodology                       | Key Metrics Measured | Key Findings (Positive)  | Key Findings (Negative / Cautionary )                           | Source(s)     |
|-----------------------|-----------------------------------|----------------------|--------------------------|-----------------------------------------------------------------|---------------|
| METR (July 2025)      | Randomized Controlled Trial (RCT) | Task Completion Time | N/A                      | <b>19% slower</b> with AI tools; strong perception-reality gap. | <sup>37</sup> |
| GitHub/Accenture (May | RCT in Enterprise                 | PRs, Build Success,  | +15% PR merge rate, +84% | Not specified.                                                  | <sup>35</sup> |

|                            |                                           |                                           |                                                                |                                                                              |    |
|----------------------------|-------------------------------------------|-------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------|----|
| 2024)                      |                                           | Satisfaction                              | successful builds, 90% higher fulfillment.                     |                                                                              |    |
| Faros AI (Sept 2023)       | Cohort Analysis                           | Lead Time, PRs, Code Quality              | <b>55% reduction in Lead Time</b> , increased PRs.             | Slight increase in "code smells".                                            | 44 |
| Mohamed et al. (July 2025) | Systematic Literature Review (37 studies) | SPACE Dimensions                          | Accelerated development, reduced code search.                  | Reduced team collaboration, inconsistent code quality, cognitive offloading. | 49 |
| UK Gov Trial (Sept 2025)   | Survey & Telemetry                        | Time Saved, Satisfaction, Acceptance Rate | <b>56 mins/day saved</b> (self-reported), 6.6/10 satisfaction. | Low acceptance rate (15.8%), only 39% committed AI code.                     | 43 |
| Stanford/NYU (2023)        | Code Analysis                             | Security Vulnerabilities                  | N/A                                                            | <b>40% of AI-generated code contained vulnerabilities.</b>                   | 53 |

|                         |                  |                                            |                                       |                                                                                                          |    |
|-------------------------|------------------|--------------------------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------------|----|
| <b>Sonar<br/>(2025)</b> | Code<br>Analysis | Security,<br>Code<br>Smells,<br>Complexity | High<br>syntactic<br>correctness<br>. | >90% of<br>issues were<br>"code<br>smells",<br>>60% of<br>vulnerabili<br>ties were<br>high-<br>severity. | 54 |
|-------------------------|------------------|--------------------------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------------|----|

The clear divergence in findings shown in Table 1 underscores the critical importance of a mixed-methods approach to measurement. Studies that rely heavily on developer sentiment and self-reported data, such as satisfaction surveys, consistently yield positive results. This is because developers' perception of productivity is strongly influenced by the immediate, tangible reduction of friction in low-level tasks like typing and searching, which fosters a powerful feeling of efficiency. However, this perception often fails to account for the new, higher-level cognitive work required for verification, debugging, and security analysis—tasks that are mentally demanding but less visible as discrete "activities."

Conversely, studies employing objective, end-to-end task timing or direct code analysis reveal significant underlying problems. Quantitative system metrics like pull request volume can also be misleading; a higher number of PRs may simply reflect more rework and lower-quality initial submissions. Therefore, relying on any single data source—be it surveys, telemetry, or code inspection—provides an incomplete and potentially biased view. A robust understanding of the true impact of LLMs requires a triangulation of data across multiple dimensions: developer sentiment (Satisfaction), system-level throughput (Activity and DORA metrics), and deep code inspection (Performance). The contradictions in the literature are not necessarily methodological flaws but are instead reflections of different facets of a complex, multi-dimensional phenomenon.

## 5. Discussion

### 5.1. Interpretation of Results: Reconciling the Productivity Paradox

The central finding of this review is the existence of a "productivity paradox," characterized by a significant divergence between developers' high perceived productivity and objective measures that show inconsistent or even negative impacts on efficiency and quality. This paradox can be explained through cognitive biases, such as the **IKEA effect** and the **sunk cost fallacy**. The considerable effort developers invest in crafting prompts, iterating on suggestions, and "chaperoning" the LLM to a correct solution may inflate their perceived value of the final output, leading them to believe the process was more efficient than it was.<sup>37</sup>

Ultimately, the impact of LLMs is best understood not as a simple amplification of existing productivity but as a fundamental **shift in the nature of developer work**. Effort is reallocated from the direct generation of code to the higher-level tasks of specification, curation, verification, and integration of AI-generated artifacts. The net effect on productivity is therefore not guaranteed; it depends entirely on the efficiency of this new, human-AI collaborative workflow and the developer's skill in critically evaluating AI output.

## 5.2. Link to Literature: Human-AI Collaboration and Knowledge Work

These findings align with broader research in the field of human-AI collaboration, which consistently demonstrates that AI tools are most effective when they complement and augment human skills rather than attempting to fully automate complex tasks.<sup>66</sup> The current generation of LLMs, often described as "stochastic parrots" for their ability to mimic patterns without true understanding, function as powerful pattern-matchers, not as genuine reasoning partners.<sup>1</sup> Their failure to grasp the deeper semantic context of a software system is the root cause of the logical and security flaws they produce.

The challenges in measuring the impact of LLMs also echo long-standing difficulties in quantifying the productivity of knowledge workers.<sup>19</sup> In knowledge work, the output is non-standardized, and quality is often more important than quantity. The introduction of LLMs reinforces the need to shift measurement focus from simple output metrics (like LOC or commits) to outcome-based metrics that capture the value, reliability, and maintainability of the software delivered.

## 5.3. Implications

The transformative nature of LLMs carries profound implications for the software engineering

profession at multiple levels.

- **For Engineering Leaders:** The primary implication is that traditional productivity measurement is no longer sufficient and can be actively misleading. Relying on activity metrics like commits or story points is particularly dangerous in the LLM era, as it may incentivize the generation of low-quality code. A balanced scorecard approach, such as the SPACE framework, is essential for a holistic view.<sup>34</sup> Furthermore, leaders must recognize that adopting LLMs necessitates a corresponding investment in robust quality assurance processes, particularly automated security scanning and a reinforced culture of rigorous code review, to mitigate the significant quality and security risks.
- **For Software Developers:** The role of the software developer is evolving. The emphasis is shifting away from being a "coder" or producer of code to being a "system architect," "problem solver," or "code curator".<sup>68</sup> The most valuable skills in this new paradigm are system design, problem decomposition, effective prompt engineering, and, most critically, the ability to rigorously validate, debug, and secure AI-generated outputs.<sup>4</sup> There is a tangible risk of skill erosion, or "deskilling," for developers who become overly reliant on these tools without cultivating a deep understanding of the underlying principles.<sup>4</sup>
- **For Tool Creators:** The next generation of AI coding assistants must evolve beyond simple code completion to address the key bottlenecks in the new workflow. Future innovation should focus on enhancing context awareness to handle large, complex codebases; integrating security and quality analysis directly into the suggestion engine; and developing features that actively support the human review and verification process, rather than simply generating more code that requires scrutiny.

## 6. Conclusion and Recommendations

### 6.1. Conclusion

The integration of Large Language Models into software development represents a disruptive and transformative change, but it is not the unqualified productivity panacea it is often portrayed to be. This systematic review reveals a complex landscape of trade-offs. While LLMs offer clear and significant benefits in automating routine tasks, accelerating the development of simple components, and enhancing developer satisfaction, these gains are



frequently counterbalanced by substantial risks to code quality, security, and end-to-end efficiency for complex tasks. The prevailing narrative of a universal productivity boost is not supported by the current body of rigorous empirical evidence. The true value of LLMs will be realized not by replacing human developers, but by augmenting them. This requires a paradigm shift in how organizations measure productivity, structure their workflows, and cultivate developer skills to navigate the challenges and harness the potential of this powerful new class of tools.

## 6.2. Recommendations for Future Research

The existing body of research, while growing rapidly, has several critical gaps that future work should address:

- **Longitudinal Studies:** The majority of current studies are short-term and exploratory in nature.<sup>49</sup> Longitudinal research is urgently needed to understand the long-term effects of LLM adoption on crucial factors like technical debt, code maintainability, and the evolution of developer skills.
- **Team-Level Analysis:** The impact of LLMs on team dynamics, collaborative practices, and collective knowledge sharing is a critically underexplored area.<sup>49</sup> Research must move beyond its current focus on individual developers to conduct team-based evaluations.
- **Contextualized Studies:** More research is required to understand how the impact of LLMs varies across different software domains (e.g., embedded systems vs. web development), project scales, and organizational cultures.

## 6.3. Recommendations for Practitioners

For organizations and developers integrating LLMs into their workflows, the evidence suggests the following best practices:

- **Adopt a "Trust, but Verify" Mindset:** Treat all AI-generated code as untrusted input from an unvetted junior developer. Rigorous human oversight, mandatory code reviews, and the implementation of automated quality and security gates are non-negotiable prerequisites for safe adoption.
- **Implement Holistic Measurement:** Discard simplistic activity metrics in favor of a balanced framework like SPACE. This will provide a more accurate picture of the true

impact of LLMs on satisfaction, performance, collaboration, and efficiency, preventing the optimization of one dimension at the expense of others.

- **Invest in Training and Culture:** Developer training should focus not just on how to use AI tools, but on how to critically evaluate their outputs, engage in effective prompt engineering, and debug AI-generated code. Foster an engineering culture that continues to value deep system understanding, first-principles problem-solving, and meticulous quality assurance over the sheer velocity of code generation.

## 7. References

*(A comprehensive list of all cited sources would be compiled in a full publication.)*

## 8. Appendices

- **Appendix A: Detailed Breakdown of the SPACE Framework Dimensions and Metrics.**
- **Appendix B: List of Primary Studies Included in the Systematic Literature Review.**

### Works cited

1. Editorial – The Use of Large Language Models in Science: Opportunities and Challenges, accessed September 15, 2025, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10485814/>
2. The Evolving Landscape of LLMs in Enterprise Applications - Scribble Data, accessed September 15, 2025, <https://www.scribbledata.io/blog/the-evolving-landscape-of-llms-in-enterprise-applications/>
3. The Transformative Influence of LLMs on Software Development & Developer Productivity, accessed September 15, 2025, <https://arxiv.org/html/2311.16429v2>
4. The Transformation of Software Development in the Age of LLMs | by Rahul Agrawal, accessed September 15, 2025, <https://medium.com/rahul-agrawal/the-transformation-of-software-development-in-the-age-of-llms-0b39dd6ad5f8>
5. Application of Large Language Models (LLMs) in Software Engineering: Overblown Hype or Disruptive Change?, accessed September 15, 2025, <https://www.sei.cmu.edu/blog/application-of-large-language-models-llms-in-software-engineering-overblown-hype-or-disruptive-change/>
6. What Are Large Language Models (LLMs)? - IBM, accessed September 15, 2025,

- <https://www.ibm.com/think/topics/large-language-models>
7. Understanding large language models: A comprehensive guide - Elastic, accessed September 15, 2025, <https://www.elastic.co/what-is/large-language-models>
  8. Large language model - Wikipedia, accessed September 15, 2025, [https://en.wikipedia.org/wiki/Large\\_language\\_model](https://en.wikipedia.org/wiki/Large_language_model)
  9. www.ibm.com, accessed September 15, 2025, <https://www.ibm.com/think/topics/large-language-models#:~:text=LLMs%20consist%20of%20multiple%20layers,specific%20parts%20of%20data%20sets.>
  10. The Role of Generative AI in Software Development Productivity: A Pilot Case Study - arXiv, accessed September 15, 2025, <https://arxiv.org/html/2406.00560v1>
  11. Cursor vs Copilot: Choosing the Best AI Coding Assistant - PromptLayer Blog, accessed September 15, 2025, <https://blog.promptlayer.com/cursor-vs-copilot-choosing-the-best-ai-coding-assistant/>
  12. Best AI Code Assistants Reviews 2025 | Gartner Peer Insights, accessed September 15, 2025, <https://www.gartner.com/reviews/market/ai-code-assistants>
  13. LLMs for Software Development - Embrace The AI-Driven Coding, accessed September 15, 2025, <https://www.bacancytechnology.com/blog/llms-for-software-development>
  14. LLMs' Reshaping of People, Processes, Products, and Society in Software Development: A Comprehensive Exploration with Early Adopters - arXiv, accessed September 15, 2025, <https://arxiv.org/html/2503.05012v1>
  15. The Role of Artificial Intelligence in Computer Science Education: A Systematic Review with a Focus on Database Instruction - MDPI, accessed September 15, 2025, <https://www.mdpi.com/2076-3417/15/7/3960>
  16. Software Productivity - UMSL, accessed September 15, 2025, [https://www.umsl.edu/~sauterv/analysis/488\\_f02\\_papers/SoftwareProductivity.html](https://www.umsl.edu/~sauterv/analysis/488_f02_papers/SoftwareProductivity.html)
  17. Tool for Measuring Productivity in Software Development Teams, accessed September 15, 2025, <https://www.mdpi.com/2078-2489/12/10/396>
  18. How to Measure Software Developer Productivity - Revelo, accessed September 15, 2025, <https://www.revelo.com/blog/tips-for-measuring-software-developer-productivity>
  19. Programming productivity - Wikipedia, accessed September 15, 2025, [https://en.wikipedia.org/wiki/Programming\\_productivity](https://en.wikipedia.org/wiki/Programming_productivity)
  20. (PDF) Defining Productivity in Software Engineering - ResearchGate, accessed September 15, 2025, [https://www.researchgate.net/publication/332926801\\_Defining\\_Productivity\\_in\\_Software\\_Engineering](https://www.researchgate.net/publication/332926801_Defining_Productivity_in_Software_Engineering)
  21. What is Developer Productivity and How to Measure It | LinearB Blog, accessed September 15, 2025, <https://linearb.io/blog/developer-productivity>